symmetry

MDPI

Article

Data-Driven Approach for Spellchecking and Autocorrection

Alymzhan Toleu [1,*], Gulmira Tolegen [1], Rustam Mussabayev [1], Alexander Krassovitskiy [1] and Irina Ualiyeva [2]

[1] Institute of Information and Computational Technologies, Almaty 050010, Kazakhstan
[2] Faculty of Information Technology, Al-Farabi Kazakh National University, Almaty 050040, Kazakhstan
* Correspondence: alymzhan.toleu@gmail.com or atoleu@iict.kz

Abstract: This article presents an approach for spellchecking and autocorrection using web data for morphologically complex languages (in the case of Kazakh language), which can be considered an end-to-end approach that does not require any manually annotated word–error pairs. A sizable web of noisy data is crawled and used as a base to infer the knowledge of misspellings with their correct forms. Using the extracted corpus, a sub-string error model with a context model for morphologically complex languages are trained separately, then these two models are integrated with a regularization parameter. A sub-string alignment model is applied to extract symmetric and non-symmetric patterns in two sequences of word–error pairs. The model calculates the probability for symmetric and non-symmetric patterns of a given misspelling and its candidates to obtain a suggestion list. Based on the proposed method, a Kazakh Spellchecking and Autocorrection system is developed, which we refer to as QazSpell. Several experiments are conducted to evaluate the proposed approach from different angles. The results show that the proposed approach achieves a good outcome when only using the error model, and the performance is boosted after integrating the context model. In addition, the developed system, QazSpell, outperforms the commercial analogs in terms of overall accuracy.

Keywords: spellchecking; autocorrection; web; data driven

check for updates

Citation: Toleu, A.; Tolegen, G.; Mussabayev, R.; Krassovitskiy, A.; Ualiyeva, I. Data-Driven Approach for Spellchecking and Autocorrection. *Symmetry* **2022**, *14*, 2261. https://doi.org/10.3390/sym14112261

Academic Editors: Lorentz JÄNTSCHI and Jan Awrejcewicz

Received: 8 September 2022
Accepted: 20 October 2022
Published: 27 October 2022

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Spellchecking is a task of automatically finding misspelled words in a document; the results of a spellchecking system could be presented to the user as underlining misspelled words. Autocorrection is a task of suggesting and substituting corresponding well-spelled words forms for misspellings. A spellchecking and autocorrection system is widely applicable for many fields, such as news article and book editing, message text and official document writing, etc. It is also applicable for prepossessing textual documents [1] and post-processing optical character recognition (OCR) [2].

For developing a spellchecking and auto-correction system for any language, it is essential to have an annotated misspellings corpus, where the information on misspelled words and their well-spelled forms are presented. Constructing this type of manually annotated corpora is time consuming and often expensive. It is hard to guarantee that the annotated corpus is balanced and covers most of the typical word errors. Kazakh is a morphologically complex language in which a single root produces hundreds and thousand of new words by attaching suffixes to it. Due to these characteristics of the language, it is impossible to store whole Kazakh word variations in a dictionary, and it covers many misspellings by manually annotating a corpus.

For instance, Figure 1 shows the variations of real misspellings which are found in the Kazakh web news automatically. It can be seen that there are many-to-one relations, which means that manually finding all those actual misspellings is time consuming and expensive. For morphologically complex language (MCL), such as Kazakh, Turkish, Finnish, etc., because of their rich variability in word forms, it is hard to manually annotate a spelling correction corpus.

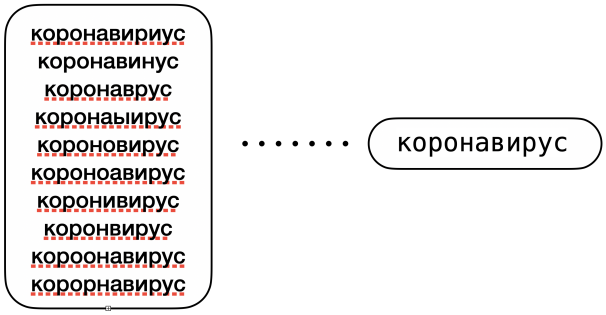


Figure 1. An example of misspelling variations of a Kazakh word "coronavirus" (misspellings on the left and their correct fix on the right).

Existing Kazakh spelling correction systems utilize an annotated corpus, and most of these systems utilize replacement rules [3], Levenshtein distances [4], and morphological analyzers plus noisy channels [5], which is summarized in Table 1. To our knowledge, the existing systems for Kazakh language are not publicly available along with the annotated corpus. One of the existing systems' limitations is that they are based on the fully string model, not a sub-string one. Facing the large variability of Kazakh words, a fully string model suffers from the high-data sparseness problem caused by the language agglutinative nature.

Table 1. Existing work for Kazakh spelling correction. FSA—finite state automata.

Studies	Method	Accuracy
Makazhanov et al. [5]	FSA + noisy channel	37%, 1-*best* 55%, 2-*best* 67%, 3-*best*
Slamova and Mukhanova [3]	Rules + Levenshtein	85.4%
Abdussaitova and Amangeldiyeva [4]	(i) Levenshtein-based; (ii) Levenshtein + classification; (iii) Naive-Bayes	(i) 62% (ii) 71.3% (iii) 89.38%

In this paper, we present a data-driven, sub-string approach for Kazakh spellchecking and autocorrection, which do not require any manually annotated corpora and any explicitly compiled dictionary of well-spelled words. Kazakh is a less-resourced language NLP-wise. The main purpose of this work is as follows: (i) to develop a sub-string spelling correction system for Kazakh, which does not require any explicitly annotated corpus and can be applied to other MCLs; (ii) to create a Kazakh word-error pairs corpus from a large noisy corpus for future use; and (iii) to produce a strong baseline work for Kazakh spelling correction, which is easy to follow.

Recent general studies on spelling correction are based on more complex neural networks, for example, Gan et al. [6] presented a spelling error correction model with a soft-masked BERT model [7] in a curriculum learning (CL) manner. Ji et al. [8] proposed a SpellBERT which is four layers of Bert, but only the half size of vanilla BERT, and can show competitive performance and produce a state-of-the-art result on the OCR dataset. The proposed competitive spelling correction approach for heavily resourced languages (such as English, Chinese, etc.) is an excellent scientific contribution. However, considering the purpose of this article and making the work fit to less being more principal, for a less-resourced language, such as Kazakh, a spelling correction model along with its dataset is more necessary.

An error model [9] is a simple and sub-string approach that has achieved good outcomes for many languages. We apply this sub-string method to Kazakh spelling correction task as a base model in this work. Then a context model of Kazakh text is

integrated into the sub-string model to improve the system's performance further. This approach requires an annotated Kazakh corpus of word–error pairs. To solve this problem, we propose a purely data-driven approach that infers the knowledge of misspelling with its correct fix from sizable noisy web data.

The proposed approach consists of five stages:

- Term filtering—instead of building a well-spelled lexicon, a term list is collected in a simple filtering manner.
- Triple inferring stage—the useful information about misspellings terms is inferred from observing the noisy web data.
- Alignment of the misspelling and the intended words; this is used to build an error model.
- To enhance the error model, we build an n-gram language model for Kazakh, which makes the model with context-appropriate corrections.
- Candidate scoring step—develop and optimize the mixture of the error model and language model; as a result, our system can detect and correct real-word substitutions, i.e., word usage and grammatical errors.

To inter triples, we introduce a context-based filtering approach that first filters terms list with frequency using the Levenshtein–Damerau algorithm to infer orthographically similar terms then uses a context-shared strategy to infer those misspellings. For a MCL (such as Kazakh), inferring misspelled words from noisy web data is a challenging task due to the agglutinative nature of language. A window-based error model is introduced, which makes an alignment between misspellings and intended words at the sub-string level, and calculates the probabilities of how likely it is that sub-strings from misspelled words transfer to sub-strings of intended words. Several experiments are conducted to evaluate the proposed approach, and the results show that QazSpell outperforms the baseline that is a commercial analog.

The rest of the paper is organized as follows: Section 1 introduces the work, and Section 2 describes the related work. Section 3 introduces the proposed system, followed by the experiments and results reported in Section 4. Finally, we conclude the work with future work in Section 5.

2. Related Work

This section provides an overview of prior work on the correction of spelling errors in general. A spelling correction system consists of three stages of processing:

- Detects a spelling error;
- Generating a set of candidates for correction;
- Ranking the candidates.

Below, we describe related early work regarding these three angles. Then, we introduce the existing studies for the Kazakh spelling correction task to compare it with this work.

2.1. Error Detection

Spellchecking is one of the typical problems in natural language processing, and one of the early works of [10] provided many solutions; most approaches are based on the use of one or more manually compiled resources.

A word is either a common word or a lower-frequency proper name, or it may be borrowed from another language. According a misspelled word belongs to a dictionary of correct words or not, word errors can be divided into two classes [10]:

- Real-word errors, where a word is misspelled, but its misspelled form is correct in the language.
- Non-word errors, where a word is misspelled and its form is not in the dictionary.

A fast lookup technique can be applied for non-word error detection, such as hash table [11] or search tree [12,13], and a misspelled word can be searched for in a dictionary. The most commonly used open-source spelling correction systems are Aspell (http://aspell.net, accessed on 29 July 2022) and Hunspell (http://hunspell.github.io/, accessed on 29

July 2022), which are used for detecting non-word errors in most of cases. Real-word errors are hard to detect than non-word errors because it requires semantic analysis of context. Mashod Rana et al. [14] proposed an autocorrection system for the Bangla language using a language model to correct a homophonic real-word error. Deorowicz and Ciura [15] claimed that creating a lexicon for all correct words could be too large and it can lead to misdetection and many real-word errors. Wang and Liao [16] projected each word in a test sentence into a high dimensional vector space in order to reveal and examine their relationships by using a conditional random field (CRF)-based detector .

Levenshtein–Damerau edit distance was introduced in the work [17] as a standard way to detect spelling errors. The noisy channel model was applied to the spelling correction task by Kernighan et al. [18] and Church and Gale [19]. The noisy channel model is a kind of Bayesian inference, as it calculates the probability of seeing a misspelled word and trains the parameters with the condition of how a word generates this misspelled word. A sub-string error model [9] was used for English spellchecking and autocorrection, which proposed to use a context window to partition misspelled and correct words, and using the partitions to calculate the probability of a partition of a source word conditioned to a partition of its candidate. It also adds a positional feature to the condition, which shows better results. Besides using the edit distance as features, solving the phonetic errors is another challenging task. Phonetic spelling errors could lead to a large edit distance with variously noisy forms. Atkinson [20] attempted to address phonetic errors by generating sound-like equivalent candidates based on phonetic algorithms. In this direction, Yang et al. [21] presented a generalized spelling correction system integrating phonetics to address phonetic errors in E-commerce search without additional latency cost. Zhang et al. [22] presented an end-to-end Chinese spelling correction model that integrates phonetic features into a language model by leveraging the powerful pre-training and fine-tuning method. Additionally, neural network-based spelling correction approaches [8,23] were proposed using sophisticated neural networks.

2.2. Candidate Generation

All words in a lexicon of correct words can be used as candidates after the detection process. To further optimize this process, it is reasonable to restrict the search space that are similar to the detected misspellings. Zhang and Zhang [24] proposed EmbedJoin, an approach to the problem of edit similarity joining, which calculates the edit similarity for two strings given a threshold value. Kinaci [25] used a character-level language model trained on a dictionary of correct words to generate a candidate list. Reffle [26] used a Levenshtein automata to efficiently determine word similarity. Yu et al. [27] provided a comprehensive survey on the methods of string similarity search and join.

2.3. Candidate Ranking

A noisy channel model [28] calculates similarity between two strings as a probability of converting one string into another. Brill and Moore [9] applied a noisy channel to produce a list of possible correction candidates and rank them by the obtained probability score. Except the ranking candidate, only considering the comparing of two strings, the context information can be applied in the ranking, which refers to the context model. It takes into account the misspelled word's context and makes the correct words appear on the top position in a list of candidates, which has semantic relevance to the identified misspelled word. Flor [29] presented an research work of using four types of contextual information for improving the accuracy of autocorrection system: (i) immediate local context; (ii) local lexical neighborhood using a very large distributional semantic model; (iii) recognizing a misspelling as an instance of a recurring word can be useful for re-ranking; and (vi) looks at context beyond the text itself.

2.4. Spelling Correction for Kazakh

For Kazakh spelling correction, the author [5] proposed a spelling correction tool for Kazakh language based on a morphological disambiguator [30]. Makazhanov et al. [5] reported the proposed method outperformed both open-source and commercial analogues, achieving the overall accuracy of 83% in generating correct suggestions. This approach requires a manually created lexicon, knowledge about the language, and an annotated training corpus. They collected more than 1800 error–correction pairs from the annotated Kazakh language corpus (KLC) [31]. The proposed method produces 37%, 55% and 67% accuracies for 1-*best*, 2-*best* and 3-*best* suggestions.

Slamova and Mukhanova [3] presented a text normalization and spelling correction method for Kazakh language by applying a set of replacement rules as regular expression patterns and keyboard errors with the Damerau distance calculation. For the 500-word test set, their correction system archived 85.4% accuracy. Abdussaitova and Amangeldiyeva [4] proposed a Kazakh text normalization study using three methods: (i) the Levenshtein-based algorithm, (ii) Levenshtein plus classification rules-based algorithm and (iii) naive-Bayes-based algorithm. The dataset was collected from a survey and Kazakh websites. The approximate size of the dataset was about 110 thousand words, and it includes the most common, ill-formed, and spoken words. The accuracies of their three methods are 62%, 71.3%, and 89.38%, respectively. Table 1 summarizes the existing studies for Kazakh spelling correction for reference. Most Kazakh spelling correction approaches require a compiled lexicon containing well-spelled words and manually annotated error–correct word pairs as the training corpus. Existing Kazakh spelling correction systems are rule-based or distance metric based, and the naive Bayes one used a fully string model, not a sub-string one. None use a context model, which plays a vital role in correcting contextual word errors in practice. The corpus collected by the previous work is not publicly available. The proposed approach in this article has several differences: (i) It does not require any manually annotated corpora and any explicitly compiled dictionary of well-spelled words. The training data are extracted from the noisy web data without the use of any manually annotated corpus. (ii) The proposed method is a purely data-driven approach that does not require any replacement rule or manually annotating the data. (iii) A sub-string model integrated with a context model is presented. Due to the agglutinative nature of the Kazakh language, using a sub-string model is reasonable, rather than using a full-string model.

3. Methodology

This section describes our methodology (with an example of Kazakh) for spellchecking and autocorrection using noisy web data without the use of any manually annotated corpus. The proposed approach consists of five steps described in the following subsections. Figure 2 shows the process of spellchecking and autocorrection.

3.1. Term List

To score the candidate list for correction, we require a list of well-spelled words. Rather than attempt to build a dictionary that contains well-spelled words; instead, we take the most frequent term observed on the web. The idea behind this attempt is that the correctly spelled words mostly appeared compared to the misspelled ones. To do this, we crawl a large Kazakh news article from the web, and then we apply several steps of pre-processing, namely by tokenization [32], removing punctuation, numbers, and special characters. We created a list of candidate words with simple frequency-based filtering by taking the most frequent term observed on the web. We do not set a large frequency threshold for this due to the agglutinative nature of the language (hundreds and thousands of new words can be produced from a single root), and the word frequency for Kazakh has a long tail effect, which means most of the words' frequencies are small and similar to each other. The obtained term list is so large ($\sim$2 million tokens), and it should contain well-spelled words but also contain misspelled words.

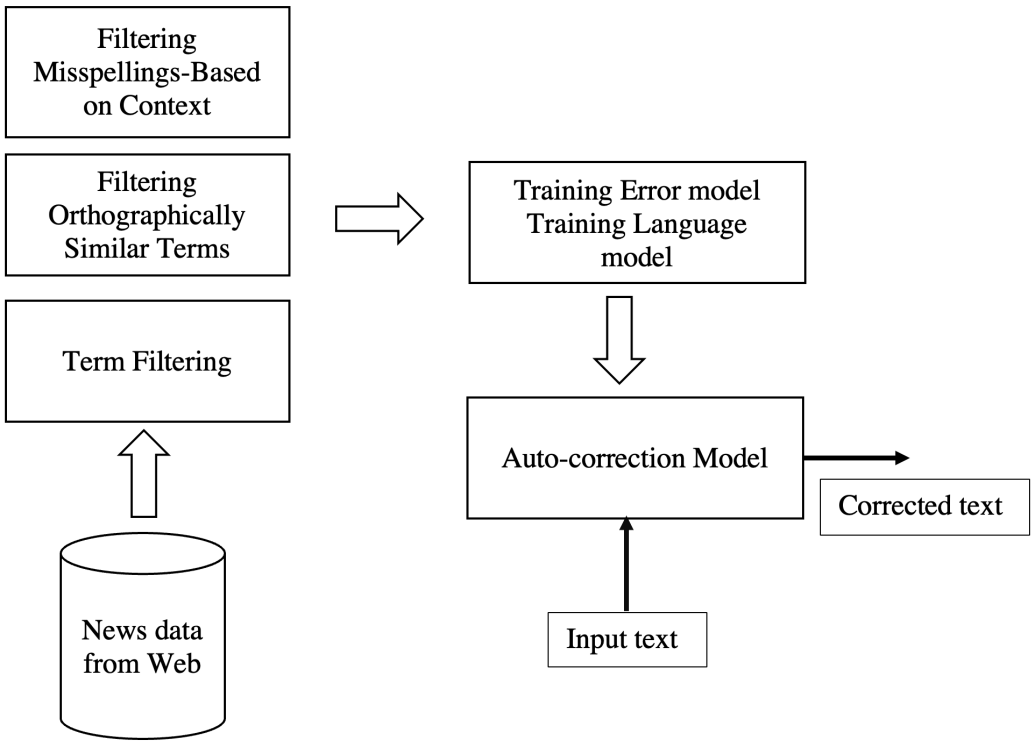


Figure 2. The process of spellchecking and autocorrection.

3.2. Using Web to Infer Misspellings

To build an error model, we require a training dataset with a set of triples (misspelled word, intended word, and count), which is also derived from the large web data. We believe the web data are ideal for inferring the triples because most of the web data are generated by users, and they contain well-spelled words and real misspelled words. In the autocorrection process, we do not directly use indented words in triples cause our method models sub-strings from well-spelled and misspelled words. To find the triples, we make two assumptions: (i) misspelled words are orthographically similar to indeed words, which is found by [17], who observed that 80% of 892 misspellings is derived from single instances of insertion, deletion, or substitution. (ii) Most words are usually well spelled on the web and can be treated as intended words.

To extract the triples, at first, we use the obtained term list. For each word in the term list, we find all other terms in the list that have small orthographic similarities. We define the similarity using the Levenshtein–Damerau edit distance, which is used with restriction in the following manner: for word length, $\leq$4, the edit distance is set to one, and if the word length ranges within 4 $\sim$ 12, the edit distance is set to two and set to three for longer words. To speed up the computation of the edit distances process, we compile the term list into a tree-based data structure that allows for efficient searching on the entire term list with a maximum edit distance. We also use multiprocessing to parallel the computations to further speed up the distance calculation.

At this stage, for each term in the list, we find a cluster of terms that are orthographically similar to it. One of the issues in the obtained misspellings is that they still contain many well-spelled words that only differ from the intended word with added suffixes. To tackle this issue, a context-shared filtering algorithm is introduced, which is based on the following two assumptions:

- Most of the time, intended words occur more in context than misspellings.
- Misspellings are fewer in occurrence in the context and have a similar context with the intended word.

We use the above assumptions to identify misspelled and intended words by filtering the triples obtained from the edit distance filtering stage. Algorithm 1 shows how to filter misspelled and intended words.

Algorithm 1 : Filtering misspelled and intended terms.

Input: a term list $T = (w', \{w\})$, a set of contexts C and a threshold th.
Output: $D = (w, \{s\})$.

for $<(w', \{w\}) \in T>$ **do**
  $\{ms\} \leftarrow \emptyset$
  for $<t_i \in \{w\}>$ **do**
    $count[i] \leftarrow C[t_i]$
  end for
  $iw \leftarrow \{w\}[\arg\max_{(count)}]$ using first assumption to find the intended word.
  for $<t_i \in \{w\}>$ **do**
    if $t_i \neq iw$ **then**
      $sharedcontext \leftarrow len(C[t_i] \cup C[iw])$ using second assumption to find the misspellings.
      if $sharedcontext \geq th$ **then**
        $\{ms\} \leftarrow t_i$
      end if
    end if
  end for
  $D \leftarrow (iw, \{ms\})$
end for

3.3. Error Model

Our spellchecking model is based on a noisy channel model of spelling errors [19]. For an observed word w, the model should generate a list of candidate words s by the following calculations:

$$P(s|w) = P(w|s) \times P(s) \tag{1}$$

where the Bayes' rule was applied and the constant denominator was dropped. We obtain the unnormalized posterior as a noisy channel model with two components: a source model $P(s)$ and a channel model $P(w|s)$.

Instead of using noisy channel model on entire words, we use a sub-string error model to calculate the probability of $P(w|s)$. To derive such a sub-string error model, we define R to be all possible adjacent partitionings of s and similarly let T be the partitioning of w, where $|R| = |T|$. The partitioning process is in one-to-one alignment and also allows the partition to be empty. For particular partitions R and T, the error model [9] estimates $P(w|s)$ as follows:

$$P(w|s) = max_{R,Ts.t.|R|=|T|} \prod_{i}^{|T|} P(T_i|R_i) \tag{2}$$

We train the error model by using the triples (misspelled word, intended word, and count) derived in the above steps and use the maximum likelihood estimates of $P(T_i|R_i)$. Figure 3 shows an example of alignment with one edit distance. It can be seen that there are symmetric and non-symmetry patterns that can be obtained from two sequences, and the probability for those patterns should be calculated by an error model.

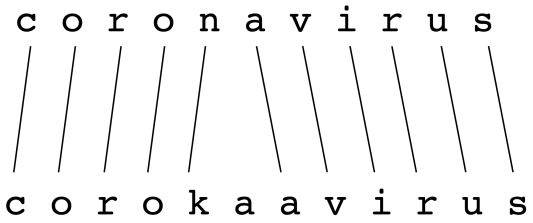


Figure 3. An example of a partition of a misspelling and its correct fix.

Two string are aligned by minimizing the edit distances based on single character insertions, deletions and substitutions. It can be seen from Figure 3, for one edit distance alignment, it corresponds to the sequence of the following symmetric and non-symmetric patterns:

$$c \to c, o \to o, r \to r, o \to o, n \to k, \varepsilon \to a, a \to a, v \to v, i \to i, r \to r, u \to u, s \to s$$

To obtain richer context information, the window size can be increased, then for the first mismatch of the above example, the model can generate the following pairs (in case of edit distance = 3):

$$n \to k, on \to ok, n \to ka, on \to oka, na \to kaa$$

For nonmatch edits, we will get similar alignments, then we can calculate the probability of them. It indicates how likely a right-side sub-string can be produced by the left-side sub-string. In practice, it can be done by counting those two sub-strings from the corpus for estimation.

In this way, we can obtain a error model which calculates the probability for misspelling words and its candidates: how likely it is that the candidate fits to the misspelled word with the sub-string model. Currently, the error model can generate candidates by calculating sub-string probabilities that only capture the word orthographic features.

3.4. Context Model

In order to detect real-word errors and to improve the error model, only capturing orthographic features is not enough; the semantic analysis of the context is required to detect and generate candidate words. Another benefit of using a language model is reducing the searching space, as there is no need to calculate the similarity score for all lexicons derived from the web.

For this purpose, we incorporate a language model to the error model. We use the n-gram language model with Stupid Back-off [33] to estimate the second part $p(s)$ of Equation (1). We use a large text corpus to train a trigram language model which can be formulated as follows:

$$P(w_1, ..., w_m) = \prod_i^m P(w_i | w_1, ..., w_{i-1}) \tag{3}$$

3.5. Ranking

How to rank the obtained candidates is an important issue in an autocorrection system, which affects the performance of the final results significantly. We score each candidate word by combining the error model's score and the language model's score.

The final score for each candidate correction is computed as follows:

$$P(s|w) = \lambda \cdot \overbrace{P(w|s)}^{\text{error model}} \times (1-\lambda) \cdot \overbrace{P(s)}^{\text{context model}} \tag{4}$$

Hyper-parameter λ is tuned during the experiments.

4. Experiment

To evaluate the QazSpell performance, we conducted a set of experiments: (i) hyper-parameter tuning experiment; (ii) investigation of the accuracy of QazSpell for x-best candidates and n-window (edit distance); (iii) a comparison experiment of QazSpell using a language model and not using it; (iv) a comparison experiment with a commercial analogue, where we compare QazSpell with the Microsoft office for Mac (MSO) version 16.56 as the baseline; and (v) an evaluation of the stability of QazSpell by increasing the test data.

4.1. Data Set

We crawled a collection of Kazakh news text from different sources, shown in Table 2. The obtained web data were pre-processed with several steps: (i) removing numbers, punctuation, emails, links and extra special characters; and (ii) applying sentence and word segmentation. The statistics of the obtained data are shown in Table 3. It can be seen that there are over 2 million tokens as the term list, and 27,228 error–correction pairs are filtered out after applying Algorithm 1.

Table 2. Crawled web data sources used in QazSpell (accessed on 3 May 2020).

'http://today.kz',
'http://vesti.kz/',
'http://www.kp.kz/',
'http://www.newsfactory.kz/',
'http://www.spik.kz/',
'https://24.kz/ru/',
'https://365info.kz/',
'https://aif-kaz.kz/',
'https://bnews.kz/ru(baigenews.kz)',
'https://forbes.kz/',
'https://inbusiness.kz/ru',
'https://kapital.kz/',
'https://kazakh-tv.kz/ru',
'https://kaztrk.kz/ru',
'https://liter.kz/',
'https://rus.azattyq.org/',
'https://sputniknews.kz/',
'https://tengrinews.kz/',
'https://www.kt.kz/',
'https://www.ktk.kz/',
'https://www.nur.kz/',
'https://www.zakon.kz/'

Table 3. Data statistics.

documents	244,277
tokens	2,043,209
triples	27,228

To evaluate the models, we randomly selected 500 words from the corpus with its left context in order to compare the system's performance under the condition of adding a language model. For each selected word, we made a set of synthetic misspelled words by randomly taking one of the following operations:

- Inserting a random character in a random position of the word;
- Substituting a random character of the word with a random character;
- Deleting a random character from the word;
- Exchanging the position of a random character with another random character.

In total, 500 error–correction pairs with their context were obtained as the test for evaluation. QazSpell was compared with Microsoft Office manually; we put 500 word error with their trigram context into Microsoft Word and manually checked if it correctly identified and corrected the misspellings.

4.2. Results

We begin with reporting the hyper-parameter tuning process; a small set of test data was chosen for this experiment. To tune a parameter $\lambda \in [0, 1]$, we heuristically choose a set of its values: $[0.1, 0.12, 0.13, 0.15, 0.2, 0.3, 0.4, 0.7]$. Figure 4 plots how the accuracy of QazSpell is performed when setting the various value of λ. The result shows that the model performs best at the value of 0.15. The model's accuracy decreases when taking larger or smaller values than that. Thus, in the following experiments, we set hyper-parameter λ to 0.15.

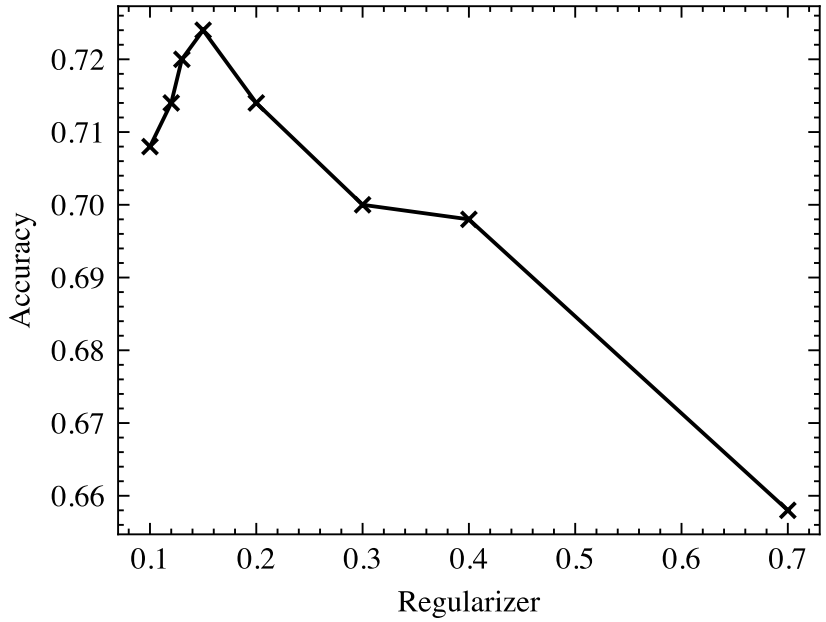


Figure 4. Accuracy results against with regularization.

Second, we analyze QazSpell by important metrics: x-*best* and n-window. The former is calculated as a percentage of the correct suggestion that appeared at the first x-positions of the ranked suggestion lists. The latter is calculated when the model is trained for different maximum context window sizes.

Table 4 shows the accuracy results of QazSpell without a language model. It can be seen that for the x-best evaluation, the model produces $\approx$ 0.65 accuracy for 1-*best*. The model gains a large improvement in 2-*best* ($\approx$11%), and 3-*best* ($\approx$5%). For rest of them, QazSpell gains minor improvements, and at 10-*best*, it reaches a high result of accuracy $\approx$93.

Table 4. Autocorrection accuracy without a language model.

x-*best*\Window	**1**	**2**	**3**	4	**5**
1	0.664	0.636	0.654	0.654	0.656
2	0.77	0.76	0.77	0.77	0.77
3	0.824	0.816	0.83	0.83	0.83
4	0.862	0.86	0.868	0.868	0.868
5	0.882	0.884	0.888	0.89	0.89
6	0.89	0.9	0.902	0.904	0.904
7	0.906	0.912	0.914	0.914	0.914
8	0.914	0.924	0.92	0.92	0.92
9	0.924	0.93	0.93	0.93	0.93
10	0.93	0.934	0.936	0.936	0.936

Table 5 shows the accuracy results of QazSpell with a language model. As we can see, at 2-*best*, QazSpell is significantly improved (≈12) compared to 1-*best*. For the rest of x-*best*, we can observe slight improvements over its previous ones. The results are different from QazSpell without a language model, and the former one has two significant gains in 2-*best* and 3-*best*, which may indicate that a language model supplements the error model. In these experiments, we did not observe a huge difference when using various value for the window.

Table 5. Autocorrection accuracy with a language model.

x-*best*\Window	**1**	**2**	**3**	**4**	**5**
1	0.796	0.786	0.786	0.786	0.788
2	0.914	0.91	0.912	0.912	0.912
3	0.938	0.936	0.94	0.942	0.942
4	0.944	0.95	0.952	0.952	0.952
5	0.948	0.952	0.952	0.952	0.952
6	0.95	0.952	0.952	0.952	0.952
7	0.954	0.954	0.954	0.954	0.954
8	0.954	0.954	0.954	0.954	0.954
9	0.958	0.958	0.958	0.958	0.958
10	0.96	0.96	0.96	0.96	0.96

Third, we compare QazSpell's performance with and without a language model. Figure 5 plots the accuracy values of QazSpell when using a language model and not using it. It can be seen that the QazSpell's performance is sharpened after using a language model, which gains ≈ 15% improvement at 1-*best* compared to the model not enhanced with a language model. With the growth of x-*best*, the accuracies tend to be close to each other, but we still observe the advantages of using a language model.

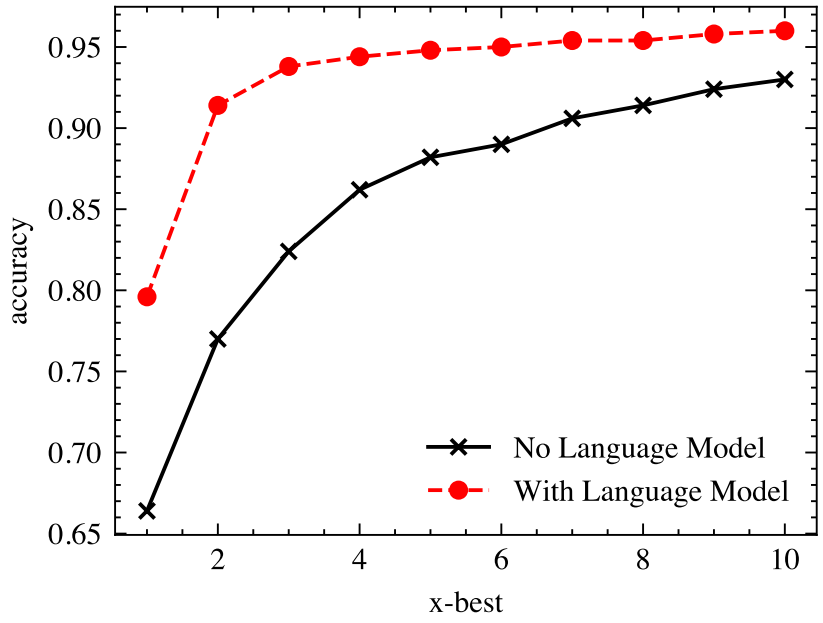


Figure 5. Spelling correction improvement when using a trigram language model.

Next, we calculate the overall accuracy for both systems, and the accuracy is calculated by ignoring the position of the correct fix in the suggestion list. The value can be considered the upper bound of QazSpell. QazSpell outperforms MSO in terms of overall accuracy; the former obtains 0.974%, and the latter obtains 0.694%, respectively.

Let us move on to compare both accuracies on x-*best* since several of the top candidates are often important in the practical use of an autocorrection system. Figure 6 shows the accuracy results of QazSpell and the baseline of MSO. Since MSO suggests three candidates at maximum, we report $[1, 2, 3]$-*best* for the comparisons. It can be seen that QazSpell outperforms MSO, whether it uses a language model or not. MSO obtains 0.584% accuracy for 1-*best*, 0.658% and 0.694% for $[2, 3]$-*best*. QazSpell obtains $\approx$ 20% $\sim$ 25% improvement over MSO.

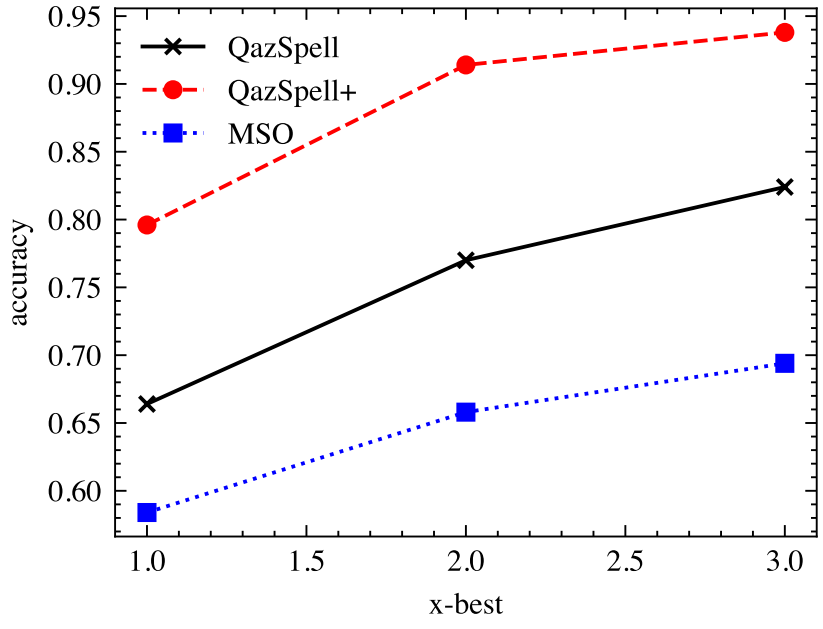


Figure 6. Accuracy results of QazSpell and MSO on x-*best*.

In the above experiments, we tested the proposed approach with a small error-correction corpus of about 500 words. In order to further evaluate the stability of QazSpell, we reassembled the test data with the same procedure described in Section 4.1; in this case, the size of the dataset is increased gradually. We collected a different-sized error-correction corpus: $D = [1000, 2000, 3000, 4000, 5000]$ with unique words, and ran the trained QazSpell to correct the misspellings. The accuracy results of QazSpell for using a language model or not using it are reported separately. This purpose is to individually evaluate the proposed error model's stability. Table 6 shows the accuracy results of QazSpell for different sized test sets without using a language model. It can be seen that for 1-*best*, the accuracy of QazSpell is varied from 0.647% (1000 words) to 0.697% (5000 words). The accuracy is not decreased when testing more data, which may indicate the robustness of the proposed error model. For other x-*best*, minor changes appear in the accuracy when the test data size is increased.

Table 7 reports the accuracy results of QazSpell with different sized test sets when adding a language model. It can be seen that with the growth of the test set, the variety of the accuracy is minimal. For the 1-*best* candidate, QazSpell achieves around 0.78%∼0.79% accuracy over the different sized test set from 1000 to 5000 words. In the 2-*best* position, the proposed approach achieves a significant improvement over the 1-*best* for all different sized-test sets. After 2-*best*, there are minor improvements that can be observed. Overall, whether the QazSpell uses a language model or not, the results indicate that the proposed approach has good stability performance when a large test set is applied.

Table 6. Evaluation of the stability of QazSpell without a language model.

x-*best*\Corpus Size	**1k**	**2k**	**3k**	**4k**	**5k**
1	0.647	0.672	0.693	0.690	0.697
2	0.771	0.788	0.805	0.805	0.816
3	0.827	0.841	0.851	0.852	0.859
4	0.859	0.874	0.881	0.882	0.889
5	0.888	0.898	0.903	0.903	0.909
6	0.899	0.908	0.913	0.913	0.919
7	0.911	0.921	0.924	0.925	0.930
8	0.924	0.93	0.933	0.933	0.938
9	0.936	0.942	0.943	0.941	0.945
10	0.944	0.947	0.947	0.945	0.949

Table 7. Evaluation of the stability of QazSpell with a language model.

x-*best*\ Corpus Size	**1k**	**2k**	**3k**	**4k**	**5k**
1	0.786	0.791	0.791	0.782	0.783
2	0.901	0.909	0.907	0.903	0.906
3	0.929	0.936	0.930	0.929	0.932
4	0.942	0.948	0.943	0.943	0.945
5	0.947	0.951	0.949	0.949	0.951
6	0.955	0.958	0.954	0.953	0.955
7	0.956	0.959	0.956	0.954	0.956
8	0.958	0.960	0.958	0.956	0.958
9	0.963	0.964	0.961	0.959	0.961
10	0.965	0.966	0.963	0.960	0.962

The speed of QazSpell was evaluated on a computer of the following environment: MacBook Pro, Apple M1 Pro, 16GB. We found that a direct aligning sub-string model works very slowly in practice, especially when there are sizable candidates that should be ranked. Instead of aligning the sequences directly, QazSpell's speed was optimized with a tree-structure-based sub-string model. With this optimization, QazSpell took 73 CPU seconds to correct the 1000 misspellings. No parallelism techniques were applied in this evaluation.

5. Conclusions

Most of the existing spelling correction systems for Kazakh are developed on an annotated word–error pair corpus, and their systems with their annotated data are not publicly available. Fully string approaches are applied in the existing Kazakh spelling correction systems, which suffer from the data sparseness problem caused by the agglutinative nature of language. Kazakh is a morphologically complex language, in which, a single root may produce hundreds and thousands of new words, considering that the Kazakh language is a less-resourced language, NLP-wise.In this article, we represent a data-driven, sub-string approach for spellchecking and autocorrection for morphologically complex languages (MCL) in the case of the Kazakh language.

The main feature of the proposed approach is that it does not require any manually annotated corpora nor any explicitly compiled dictionary of well-spelled words. Second, unlike the existing Kazakh spelling correction system that applies a fully string model based on replacement rules, Levenshtein distances and noisy channel, the proposed approach is a sub-string error model integrated with a context model, which can be a new baseline for the Kazakh spelling correction task. Third, a corpus of Kazakh word–error pairs from a large noisy corpus is created for future use.

The triples of <misspelled words, intended words, count> from a sizable noisy web data are inferred and used to train a sub-string error model. A context model is trained on a large corpus and integrated with the sub-string model to further improve its performance. A set of experiments is conducted to evaluate the proposed approach from different angles: (i) tuning the regularization of the error model and the context model by a hyper-parameter; (ii) evaluating the proposed model of a sub-string model and sub-string +context model in terms of accuracy at x-best, independently; (iii) comparing the proposed model with a commercial analogy; (iv) evaluating the stability of the proposed approach by increasing the size of test set; (v) testing the running speed of QazSpell. Experimental results show that the sub-string model achieves comparable results when comparing with the existing Kazakh spelling correction system (because there is no shared test set, this is only for reference). The sub-string error model gains a significant improvement when using a context model, and it outperforms a commercial analogue. The stability evaluation result shows that QazSpell results are stable when increasing the test set size, whether it uses a context model or not. Without using any parallelism techniques, only using one CPU core, QazSpell took 73 CPU seconds to process the 1000 misspelled words.

Although these results satisfy the study's purpose, we found several limitations of QazSpell: (i) Because the word–error pairs derived from a sizable noisy web data, there are false misspellings that exist in the corpus (this is reasonable due the agglutinative nature of languages), which bring noisy information to the model. (ii) The extracted triples and the tokens are still in the surface word forms, and the current sub-string model may not fully overcome the sparseness problem caused by language. One possible solution is using a heuristic method to detect the lemma border to a misspelling then apply two separate sub-string models to the lemma and suffixes part, then integrate to a global error model.

There are several lines of possible future work: (i) propose a cascade sub-string error model to overcome the second limitation of QazSpell mentioned above; (ii) propose a neural network-based (several architectures will be applied, such as CNN [34,35], long short-term memory [36], Bert [7], etc.) error model to further improve the performance; (iii) apply

different methods to identify more sophisticated errors, such as grammar, cognitive spelling errors, etc.

Author Contributions: Conceptualization A.T., G.T., R.M., A.K. and I.U.; methodology A.T., G.T., R.M. A.K. and I.U.; software A.T. and G.T.; validation A.T. and R.M.; formal analysis A.T. and G.T.; investigation G.T., A.K. and I.U.; resources R.M.; data curation A.T. and G.T.; writing—original draft preparation A.T. and G.T.; writing—review and editing A.T. and G.T.; visualization A.T. and G.T.; supervision R.M.; project administration R.M.; funding acquisition R.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Committee of Science of Ministry of Education and Science of the Republic of Kazakhstan under the grant AP09259324.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Open source web data listed in Table 2 were used to conduct the experiments.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Tolegen, G.; Toleu, A.; Zheng, X. Named Entity Recognition for Kazakh using conditional random fields. In Proceedings of the 4 th International Conference on Computer Processing of Turkic Languages TurkLang, Bishkek, Kyrgyzstan, 24–26 August 2016; pp. 118–127.
2. Sporici, D.; Cușnir, E.; Boiangiu, C.A. Improving the Accuracy of Tesseract 4.0 OCR Engine Using Convolution-Based Preprocessing. *Symmetry* **2020**, *12*, 715. [CrossRef]
3. Slamova, G.; Mukhanova, M. Text Normalization and Spelling Correction In Kazakh Language. In Proceedings of the AIST, Moscow, Russia, 5–7 July 2018.
4. Abdussaitova, A.; Amangeldiyeva, A. Normalization of Kazakh Texts. In Proceedings of the Student Research Workshop Associated with RANLP 2019, Varna, Bulgaria, 2–4 September 2019; INCOMA Ltd.: Varna, Bulgaria, 2019; pp. 1–6. [CrossRef]
5. Makazhanov, A.; Makhambetov, O.; Sabyrgaliyev, I.; Yessenbayev, Z. Spelling Correction for Kazakh. In Proceedings of the Computational Linguistics and Intelligent Text Processing, Kathmandu, Nepal, 6–12 April 2014; Gelbukh, A., Ed.; Springe: Berlin/Heidelberg, Germany, 2014; pp. 533–541.
6. Gan, Z.; Xu, H.; Zan, H. Self-Supervised Curriculum Learning for Spelling Error Correction. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Online, 7–11 November 2021; Association for Computational Linguistics: Punta Cana, Dominican Republic, 2021; pp. 3487–3494. [CrossRef]
7. Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, MN, USA, 2–7 June 2019; Association for Computational Linguistics: Minneapolis, MN, USA, 2019; Volume 1 (Long and Short Papers), pp. 4171–4186. [CrossRef]
8. Ji, T.; Yan, H.; Qiu, X. SpellBERT: A Lightweight Pretrained Model for Chinese Spelling Check. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Online, 7–11 November 2021; Association for Computational Linguistics: Punta Cana, Dominican Republic, 2021; pp. 3544–3551. [CrossRef]
9. Brill, E.; Moore, R.C. An Improved Error Model for Noisy Channel Spelling Correction. In Proceedings of the 38th Annual Meeting of the Association for Computational Linguistics, Hong Kong, China, 1–8 October 2000; Association for Computational Linguistics: Hong Kong, China, 2000; pp. 286–293. [CrossRef]
10. Kukich, K. *Techniques for Automatically Correcting Words in Text*; Association for Computing Machinery: New York, NY, USA, 1992; Volume 24, pp. 377–439. [CrossRef]
11. Miangah, T.M. FarsiSpell: A spell-checking system for Persian using a large monolingual corpus. *Lit. Linguist. Comput.* **2013**, *29*, 56–73. [CrossRef]
12. Shang, H.; Merrettal, T. Tries for approximate string matching. *IEEE Trans. Knowl. Data Eng.* **1996**, *8*, 540–547. [CrossRef]
13. Pal, U.; Kundu, P.K.; Chaudhuri, B.B. OCR error correction of an inflectional indian language using morphological parsing. *J. Inf. Sci. Eng.* **2000**, *16*, 903–922.
14. Mashod Rana, M.; Tipu Sultan, M.; Mridha, M.F.; Khan, M.E.A.; Ahmed, M.M.; Hamid, M.A. Detection and Correction of Real-Word Errors in Bangla Language. In Proceedings of the 2018 International Conference on Bangla Speech and Language Processing (ICBSLP), Sylhet, Bangladesh, 21–22 September 2018; pp. 1–4. [CrossRef]
15. Deorowicz, S.; Ciura, M. Correcting Spelling Errors by Modelling Their Causes. *Int. J. Appl. Math. Comput. Sci.* **2005**, *15*, 275–285.

16. Wang, Y.R.; Liao, Y.F. Word Vector/Conditional Random Field-based Chinese Spelling Error Detection for SIGHAN-2015 Evaluation. In Proceedings of the Eighth SIGHAN Workshop on Chinese Language Processing, Beijing, China, 30–31 July 2015; Association for Computational Linguistics: Beijing, China, 2015; pp. 46–49. [CrossRef]
17. Mays, E.; Damerau, F.J.; Mercer, R.L. Context based spelling correction. *Inf. Process. Manag.* **1991**, *27*, 517–522. [CrossRef]
18. Kernighan, M.D.; Church, K.W.; Gale, W.A. A Spelling Correction Program Based on a Noisy Channel Model. In Proceedings of the COLING 1990 Volume 2: Papers presented to the 13th International Conference on Computational Linguistics, Helsinki, Finland, 20–25 August 1990.
19. Church, K.W.; Gale, W.A. Probability scoring for spelling correction. *Stat. Comput.* **1991**, *1*, 93–103. [CrossRef]
20. Atkinson, K. Gnu Aspell 0.60.4. 2006. Available online: http://aspell.net/ (accessed on 8 September 2022).
21. Yang, F.; Garakani, A.B.; Teng, Y.; Gao, Y.; Liu, J.; Deng, J.; Sun, Y. Spelling Correction using Phonetics in E-commerce Search. In Proceedings of the Fifth Workshop on e-Commerce and NLP (ECNLP 5), Dublin, Ireland, 26–28 May 2022; Association for Computational Linguistics: Dublin, Ireland, 2022; pp. 63–67. [CrossRef]
22. Zhang, R.; Pang, C.; Zhang, C.; Wang, S.; He, Z.; Sun, Y.; Wu, H.; Wang, H. Correcting Chinese Spelling Errors with Phonetic Pre-training. In Proceedings of the Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021, Stroudsburg, PA, USA, 7–11 November 2021; Association for Computational Linguistics: Stroudsburg, PA, USA, 2021; pp. 2250–2261. [CrossRef]
23. Zhang, S.; Huang, H.; Liu, J.; Li, H. Spelling Error Correction with Soft-Masked BERT. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, online, 5–10 July 2020; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 882–890. [CrossRef]
24. Zhang, H.; Zhang, Q. EmbedJoin: Efficient Edit Similarity Joins via Embeddings. In Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '17, Halifax, NS, Canada, 13–17 August 2017; Association for Computing Machinery: New York, NY, USA, 2017; pp. 585–594. [CrossRef]
25. Kinaci, A.C. Spelling Correction Using Recurrent Neural Networks and Character Level N-gram. In Proceedings of the 2018 International Conference on Artificial Intelligence and Data Processing (IDAP), Malatya, Turkey, 28–30 September 2018 pp. 1–4;
26. Reffle, U. Efficiently Generating Correction Suggestions for Garbled Tokens of Historical Language. *Nat. Lang. Eng.* **2011**, *17*, 265–282. [CrossRef]
27. Yu, M.; Li, G.; Deng, D.; Feng, J. String similarity search and join: A survey. *Front. Comput. Sci.* **2015**, *10*, 399–417. [CrossRef]
28. Shannon, C.E. A Mathematical Theory of Communication. *Bell Syst. Tech. J.* **1948**, *27*, 379–423. [CrossRef]
29. Flor, M. Four types of context for automatic spelling correction. *Trait. Autom. Langues* **2012**, *53*, 61–99.
30. Toleu, A.; Tolegen, G.; Makazhanov, A. Character-Aware Neural Morphological Disambiguation. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, Vancouver, BC, Canada, 30 July–4 August 2017; Association for Computational Linguistics:Vancouver, BC, Canada, 2017; Volume 2 (Short Papers), pp. 666–671.
31. Makhambetov, O.; Makazhanov, A.; Yessenbayev, Z.; Matkarimov, B.; Sabyrgaliyev, I.; Sharafudinov, A. Assembling the Kazakh Language Corpus. In Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, Seattle, WA, USA, 18–21 October 2013; Association for Computational Linguistics: Seattle, WA, USA, 2013; pp. 1022–1031.
32. Toleu, A.; Tolegen, G.; Makazhanov, A. Character-based Deep Learning Models for Token and Sentence Segmentation. In Proceedings of the 5th International Conference on Turkic Languages Processing (TurkLang 2017), Kazan, Russia, 18–21 October 2017.
33. Brants, T.; Popat, A.C.; Xu, P.; Och, F.J.; Dean, J. Large Language Models in Machine Translation. In Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL), Prague, Czech Republic, 28–30 June 2007; Association for Computational Linguistics: Prague, Czech Republic, 2007; pp. 858–867.
34. Jayanthi, S.M.; Pruthi, D.; Neubig, G. NeuSpell: A Neural Spelling Correction Toolkit. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Online, 16–20 November 2020; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 158–164. [CrossRef]
35. Pariwat, T.; Seresangtakul, P. Multi-Stroke Thai Finger-Spelling Sign Language Recognition System with Deep Learning. *Symmetry* **2021**, *13*, 262. [CrossRef]
36. Sutskever, I.; Vinyals, O.; Le, Q.V. Sequence to Sequence Learning with Neural Networks. In Proceedings of the 27th International Conference on Neural Information Processing Systems, NIPS'14, Montreal, QC, Canada, 8–13 December 2014; MIT Press: Cambridge, MA, USA, 2014; Volume 2, pp. 3104–3112.